

可視化されたオブジェクトの編集による 対話的プログラム合成手法 REFSYN の提案

東京科学大学 情報理工学院 数理・計算科学系

21B30959 吉尾拓真

指導教員 増原英彦 教授

1 はじめに

プログラム合成とは、与えられた仕様を満たすプログラムを探索・生成する技術である。既存の合成手法には論理述語 [5, 6] の他、関数の入出力 (*Programming by Example, PBE*) [1, 2] やユーザーの操作 (*Programming by Demonstration, PBD*) [4] を仕様を使う手法が存在する。PBE はユーザーにプログラミング知識を要求しない点で有用であり、Excel [7] への実装で注目を集めている。

PBE プログラム合成の研究領域の課題として、**破壊的更新**を含む関数の効率的な合成の困難性が知られている。RbSyn [8] は多量の型を用いた仕様を与えることでプログラム空間を効率的に探索する手法を取っている。しかし、この仕様は PBE のような単純な入出力ではなく、複雑な型・エフェクト・ユニットテストの記述を前提とし、ユーザーにプログラミング知識を要求する問題がある。

この課題の解決のため、本研究では対話的プログラム合成手法 REFSYN を提案する。REFSYN の特徴は、ユーザーの簡便性のために、参照関係を可視化するライブプログラミング環境 Kanon [3] のテスト機能を仕様記述に用いる点である。GUI 上のグラフ操作から得られる情報を利用することで、既存の PBE 合成手法では困難な、データ構造に対する破壊的更新を含む関数の効率的な合成を目指す。

2 本研究の背景

2.1 Kanon のグラフ操作を用いたテスト機能

Kanon [3] はオブジェクトの参照関係をグラフとして可視化するライブプログラミング環境である。実行時情報を収集しカーソル位置に応じたグラフを表示することで、実行状態をユーザーに提供している。また、メソッド呼び出し前のグラフをユーザーが操作することで、不完全なメソッドに対するメソッド呼び出しの入出力のテストを設定できる。

双方向リスト末尾に要素を追加する `add` の合成を例に Kanon の動作を説明する。図 1 は `list` に対し 2 回 `add` を呼び出すプログラムである。テスト設定前にエディターの 4 行目末尾にカーソルを置くと、図 2 左が描画される。

ユーザーは、各メソッド呼び出しに対し、グラフを操作

```
1 class Node{ add(arg){...} }  
2 var list = new Node();  
3 list.val = 2;  
4 list.add(0); // (0)はテストが設定されていることを示す  
5 list.add(3);
```

図 1: JavaScript の Node の定義と `add` メソッド呼び出し

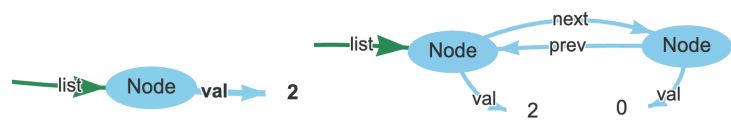


図 2: 図 1 の (左) テスト設定前 (右) 後の Kanon の可視化

してテストの期待値を設定する。例えば `list.add(0)` に対し、ユーザーはオブジェクトの追加や参照の追加・張り替えを行うことで図 2 左から図 2 右へ編集できる。Kanon はこのグラフ操作を**操作列**として記録する。

2.2 PBE プログラム合成の課題

効率的な PBE 合成にはプログラム探索空間の枝狩りが必要な一方で、既存の枝狩り手法は破壊的更新を含むプログラムの PBE 合成には適用できない。純粋関数を対象とする既存研究 [2] では、**観測的等価性**を用いた枝狩りが効率性に強く寄与することが知られている。この枝狩り手法では、より大きな式の探索中に評価結果が同じ部分式を同一視することで探索空間の削減を達成する。一方で破壊的更新を考慮すると、副作用が異なる部分式の選択は式全体の評価結果 (及び PBE 仕様の満足性) を変更しうるため、このような同一視はできない。したがって、効率的な PBE 合成のために新たな枝狩り手法が必要となる。

3 REFSYN による対話的 PBE プログラム合成

■概要 REFSYN は Kanon 上のユーザー操作を用い、破壊的更新を含むプログラムの PBE 合成問題を純粋関数の合成問題へと帰着するヒューリスティクスを用いる。破壊的更新を含む部分式を事前に探索空間から取り除き、残りの純粋な部分式を独立に合成することで、破壊的更新の存在下でも効率的な合成が期待できる。また、本手法ではユーザーは Kanon のテスト機能のみを通じて REFSYN に仕様を与えるので、特別なプログラミング知識を必要としない。

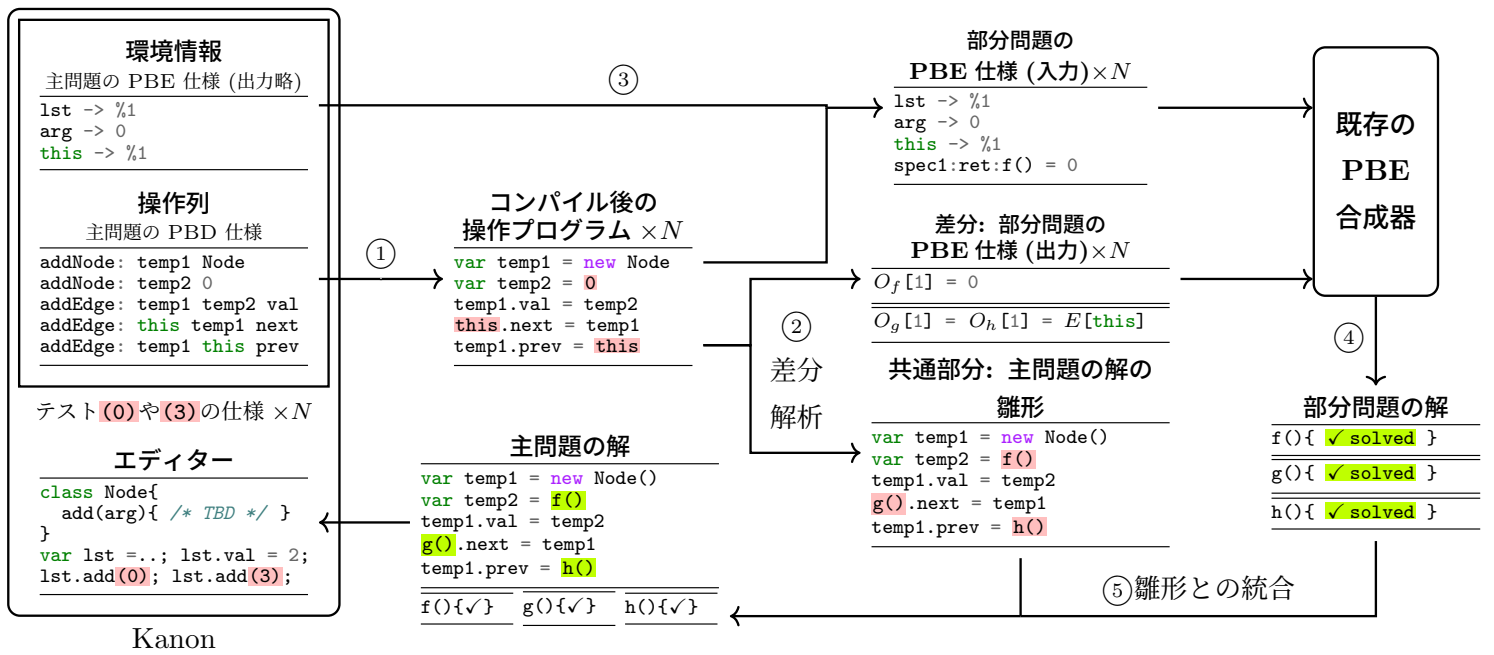


図 3: RefSyn の合成手順。 N はテスト数、 $E(p)$ は p の評価結果、 O は PBE 仕様 (出力) の配列、 $\%n$ は環境中の参照。

■**用例** RefSyn を使用するには、2.1 節で述べたように、Kanon のテスト機能を用いる。RefSyn はメソッド呼び出しテストの入出力 (PBE 仕様)・操作列 (PBD 仕様)・環境情報を受け取り、図 1 内の... に合成結果を出力する。

プログラム合成は探索中に仕様を満たした最初のプログラムを出力するため、仕様が少ないときの合成結果はユーザーの望むプログラムと異なることがある。RefSyn では、ユーザーは図 1 の 5 行目のメソッド呼び出しに追加のテストを与えることで、対話的に add の合成結果を改善できる。

4 RefSyn の合成アルゴリズム

図 3 に RefSyn の合成アルゴリズムを示す。RefSyn は③までの処理でメソッドの入出力 (PBE 仕様)・操作列 (PBD 仕様)・環境情報を入力として主問題を純粋関数の部分 PBE 合成問題へと帰着し、続く⑤までの処理で主問題のメソッド定義を合成する。

まず、①各仕様の操作列を破壊的更新を含むプログラムにコンパイルし、②コンパイル後の操作列の共通部分と差分をとる。共通部分は破壊的更新を含み、この後の処理で主問題の解の雛形として使用される。また、差分 (図中 0, this, this) はその評価値を純粋関数 (図中 f, g, h) の PBE 仕様 (出力) と見做す新たな部分問題の作成に使用される。次に③部分問題の PBE 仕様 (入力) を得るため①で得られたプログラムを実行し、環境情報を生成する。最後に④得られた PBE 仕様から部分問題を既存合成器で解き、⑤合成結果を雛形に当てはめることで主問題の合成結果を得る。

5 提案アルゴリズムの評価

RefSyn の部分問題分割の機能性を評価する。破壊的更新を含む 5 種類のプログラムを用い、純粋関数の部分問題へと分割できることを確認した。

6 結論, 関連研究, 今後の課題

本研究は対話的 PBE 合成手法 RefSyn を提案した。今後実装と、合成効率・仕様の記述容易性の観点での RbSyn との比較評価を計画している。

■**実装計画** 図 3 中③までが RefSyn の実装であり、④⑤は RefSyn を合成器として利用可能にするための実装である。④として Escher [2] の採用を予定している。合成器のレスポンスの即時性の担保のため、コンパイラターゲットに WebAssembly を持つ言語での実装を予定している。

■**PBE プログラム合成** SnipPy [1] は本研究同様の対話的な PBE 合成器である。SnipPy ではユーザーに入出力仕様を一行ずつ与えさせることで、対話性と探索空間の削減を実現する。Escher [2] はユーザーが提供した PBE 仕様から、条件式と再帰を含むプログラムを合成する帰納的アルゴリズムであり、観測的等価性を用いた効率的な合成を行う。しかし、これらはいずれも破壊的更新には対応していない。

■**今後の課題** 現在のアルゴリズムはユーザーの操作に幾つかの仮定を置いている。例えばユーザーの操作は合成対象のプログラムの想定する順番・数と同じでなければならない。これらの制約の解消は今後の課題である。

参考文献

- [1] Kasra Ferdowsifard et al. *Small-Step Live Programming by Example*. (UIST'20)
- [2] Aws Albarghouthi et al. *Recursive Program Synthesis*. (CAV'13)
- [3] Akio Oka et al. *Live, synchronized, and mental map preserving visualization for data structure programming*. (Onward'18)
- [4] Tessa Lau. *Programming by demonstration: a machine learning approach*. (PhD thesis)
- [5] Zohar Manna et al. *A deductive approach to program synthesis*. (TOPLAS'80)
- [6] Zohar Manna et al. *Synthesis of communicating processes from temporal logic specifications*. (TOPLAS'84)
- [7] Sumit Gulwani et al. *Automating string processing in spreadsheets using input-output examples*. (POPL'11)
- [8] Sankha Narayan Guria et al. *RbSyn: type- and effect-guided program synthesis*. (PLDI'21)